(

inicio( ) ( ) ( ) ( ) ( ) ( )

instrucciones( )

functions func factorial( ) ( ) ( )

n( )

type int

valRet( )

type int

block { }

( ) ( )

instrucciones( )

sIf if( ) ( )

expr( ) <=( )

expr n expr 1

block { }

( )

instrucciones( )

retorno return( )

expr 1

instrucciones( )

retorno return( )

expr( ) *( )

expr n expr( )

varCallStatement factorial( )

( )

expr( ) -( )

expr n expr 1

instrucciones( )

functions func fibonacci( ) ( ) ( )

n( )

type int

valRet( )

type int

block { }

( ) ( )

instrucciones( )

sIf if( ) ( )

expr( ) <=( )

expr n expr 1

block { }

( )

instrucciones( )

retorno return( )

expr n

instrucciones( )

retorno return( )

expr( ) +( )

expr( )

varCallStatement fibonacci( )

( )

expr( ) -( )

expr n expr 1

expr( )

varCallStatement fibonacci( )

( )

expr( ) -( )

expr n expr 2

instrucciones( )

functions func hanoi( ) ( )

n( ) , origen( ) , auxiliar( ) , destino( )

type int type string type string type string

block { }

( ) ( ) ( ) ( )

instrucciones( )

sIf if( ) ( )

expr( ) ==( )

expr n expr 1

block { }

( ) ( ) ( )

instrucciones( )

imprimir println( )

( )

expr "Mover disco 1 de "

instrucciones( )

imprimir println( )

( ) ,( )

expr origen expr destino

instrucciones( )

retorno return

instrucciones( )

varCallStatement hanoi( )

( ) ,( ) ,( ) ,( )

expr( ) -( )

expr n expr 1

expr origen expr destino expr auxiliar

instrucciones( )

imprimir println( )

( ) ,( ) ,( ) ,( ) ,( ) ,( )

expr "Mover disco" expr n expr "de" expr origen expr "a" expr destino

instrucciones( )

varCallStatement hanoi( )

( ) ,( ) ,( ) ,( )

expr( ) -( )

expr n expr 1

expr auxiliar expr origen expr destino

instrucciones( )

functions func parse_float( ) ( ) ( )

s( )

type string

valRet( )

type float64

block { }

( ) ( ) ( )

instrucciones( )

sIf if( ) ( )

expr( ) ==( )

expr s expr "123.45"

block { }

( )

instrucciones( )

retorno return( )

expr 123.45

instrucciones( )

sIf if( ) ( )

expr( ) ==( )

expr s expr "123"

block { }

( )

instrucciones( )

retorno return( )

expr 123.0

instrucciones( )

retorno return( )

expr 0.0

instrucciones( )

functions func sumar( ) ( ) ( )

a( ) , b( )

type int type int

valRet( )

type int

block { }

( )

instrucciones( )

retorno return( )

expr( ) +( )

expr a expr b

instrucciones( )

functions func main( )

) (block { }

( ) ( )

instrucciones( )

varDcl mut puntos :=( )

expr 0

instrucciones( )

imprimir println( )

( )

expr "=== Archivo de prueba de funciones ==="

instrucciones( )

varDcl mut puntos_simples :=( )

expr 0

instrucciones( )

imprimir println( )

( )

expr "\n==== Funciones no recursivas sin parámetros ===="

instrucciones( )

varDcl numero :=( )

expr 42

instrucciones( )

imprimir println( )

( ) ,( )

expr "El número obtenido es: " expr numero

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr numero expr 42

block { }

( ) ( )

instrucciones( )

varAsign puntos_simples +=( )

expr 1

instrucciones( )

imprimir println( )

( )

expr "OK Funciones sin parámetros: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Funciones sin parámetros: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "---------- jamas declaro numero asi que jamas entra en el if"

instrucciones( )

varDcl mut puntos_param :=( )

expr 0

instrucciones( )

imprimir println( )

( )

expr "\n==== Funciones no recursivas con parámetros ===="

instrucciones( )

varDcl resultado :=( )

expr( )

varCallStatement sumar( )

( ) ,( )

expr 10 expr 20

instrucciones( )

imprimir println( )

( )

expr "La suma de 10 y 20 es:"

instrucciones( )

imprimir println( )

( )

expr resultado

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr resultado expr 30

block { }

( ) ( )

instrucciones( )

varAsign puntos_param +=( )

expr 2

instrucciones( )

imprimir println( )

( )

expr "OK Función con parámetros: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Función con parámetros: incorrecto"

instrucciones( )

varDcl mut puntos_recur :=( )

expr 0

instrucciones( )

imprimir println( )

( )

expr "\n==== Funciones recursivas ===="

instrucciones( )

varDcl fact :=( )

expr( )

varCallStatement factorial( )

( )

expr 5

instrucciones( )

imprimir println( )

( ) ,( )

expr "Factorial de 5: " expr fact

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr fact expr 120

block { }

( ) ( )

instrucciones( )

varAsign puntos_recur +=( )

expr 2

instrucciones( )

imprimir println( )

( )

expr "OK factorial: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X factorial: incorrecto"

instrucciones( )

varDcl fibo :=( )

expr( )

varCallStatement fibonacci( )

( )

expr 10

instrucciones( )

imprimir println( )

( ) ,( )

expr "Fibonacci de 10: " expr fibo

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr fibo expr 55

block { }

( ) ( )

instrucciones( )

varAsign puntos_recur +=( )

expr 2

instrucciones( )

imprimir println( )

( )

expr "OK fibonacci: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X fibonacci: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\nTorres de Hanoi:"

instrucciones( )

varCallStatement hanoi( )

( ) ,( ) ,( ) ,( )

expr 3 expr "A" expr "B" expr "C"

instrucciones( )

varAsign puntos_recur +=( )

expr 2

instrucciones( )

varDcl mut puntos_atoi :=( )

expr 0

instrucciones( )

imprimir println( )

( )

expr "\n==== strconv.Atoi ===="

instrucciones( )

varDcl val :=( )

expr Atoi( )

( )

expr "123"

instrucciones( )

imprimir println( )

( )

expr "123 convertido a int: "

instrucciones( )

imprimir println( )

( )

expr val

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr val expr 123

block { }

( ) ( )

instrucciones( )

varAsign puntos_atoi +=( )

expr 1

instrucciones( )

imprimir println( )

( )

expr "OK strconv.Atoi: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X strconv.Atoi: incorrecto"

instrucciones( )

varDcl mut puntos_parse :=( )

expr 0

instrucciones( )

imprimir println( )

( )

expr "\n==== ParseFloat ===="

instrucciones( )

varDcl val1 :=( )

expr parseFloat( )

( )

expr "123.45"

instrucciones( )

varDcl val2 :=( )

expr parseFloat( )

( )

expr "123"

instrucciones( )

imprimir println( )

( ) ,( )

expr "\"123.45\" convertido a float64: " expr val1

instrucciones( )

imprimir println( )

( ) ,( )

expr "\"123\" convertido a float64: " expr val2

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) ==( )

expr val1 expr 123.45

expr( ) ==( )

expr val2 expr 123.0

block { }

( ) ( )

instrucciones( )

varAsign puntos_parse +=( )

expr 1

instrucciones( )

imprimir println( )

( )

expr "OK ParseFloat: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X ParseFloat: incorrecto"

instrucciones( )

varDcl mut puntos_type :=( )

expr 0

instrucciones( )

imprimir println( )

( )

expr "\n==== reflect.typeOf ===="

instrucciones( )

varDcl t1 :=( )

expr typeOf( )

( )

expr 42

instrucciones( )

varDcl t2 :=( )

expr typeOf( )

( )

expr 3.14

instrucciones( )

varDcl t3 :=( )

expr typeOf( )

( )

expr "Hola"

instrucciones( )

varDcl t4 :=( )

expr typeOf( )

( )

expr true

instrucciones( )

varDcl t5 :=( )

expr typeOf( )

( ) , 2 , 3]

expr 1

instrucciones( )

varDclSlice numeros_aux( ) ( ) ( ) { }

assign := nuevoSlice [] type int ( )

contenidoSlice( ) ,( ) ,( )

expr 1 expr 2 expr 3

instrucciones( )

varAsign t5 =( )

expr typeOf( )

( )

expr numeros_aux

instrucciones( )

imprimir println( )

( )

expr "Tipo de 42:"

instrucciones( )

imprimir println( )

( )

expr t2

instrucciones( )

imprimir println( )

( )

expr "Tipo de 3.14:"

instrucciones( )

imprimir println( )

( )

expr t2

instrucciones( ) instrucciones( )

imprimir println( )

( )

expr "Tipo de "

instrucciones Hola ":"

instrucciones( )

imprimir println( )

( )

expr t3

instrucciones( )

imprimir println( )

( )

expr "Tipo de true:"

instrucciones( )

imprimir println( )

( )

expr t4

instrucciones( )

imprimir println( )

( )

expr "Tipo de [1, 2, 3]:"

instrucciones( )

imprimir println( )

( )

expr t5

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) ==( )

expr t1 expr "int"

expr( ) ==( )

expr t2 expr "float64"

expr( ) ==( )

expr t3 expr "string"

expr( ) ==( )

expr t4 expr "bool"

expr( ) ==( )

expr t5 expr "[]int"

block { }

( ) ( )

instrucciones( )

varAsign puntos_type +=( )

expr 1

instrucciones( )

imprimir println( )

( )

expr "OK typeOf: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X typeOf: incorrecto"

instrucciones( )

varAsign puntos =( )

expr( ) +( )

expr( ) +( )

expr( ) +( )

expr( ) +( )

expr( ) +( )

expr puntos_simples expr puntos_param

expr puntos_recur

expr puntos_atoi

expr puntos_parse

expr puntos_type

instrucciones( )

imprimir println( )

( )

expr "\n=== Tabla de Resultados ==="

instrucciones( )

imprimir println( )

( )

expr "| Funciones con parámetros | / 2"

instrucciones( )

imprimir println( )

( )

expr puntos_param

instrucciones( )

imprimir println( )

( )

expr "| Funciones recursivas | / 6"

instrucciones( )

imprimir println( )

( )

expr puntos_recur

instrucciones( )

imprimir println( )

( )

expr "| Atoi | / 1"

instrucciones( )

imprimir println( )

( )

expr puntos_atoi

instrucciones( )

imprimir println( )

( )

expr "| ParseFloat | / 1"

instrucciones( )

imprimir println( )

( )

expr puntos_parse

instrucciones( )

imprimir println( )

( )

expr "| typeOf | / 1"

instrucciones( )

imprimir println( )

( )

expr puntos_type

instrucciones( )

imprimir println( )

( )

expr "| TOTAL | / 12"

instrucciones( )

imprimir println( )

( )

expr puntos

)